КРИПТОГРАФІЯ КОМП’ЮТЕРНИЙ ПРАКТИКУМ №3

Криптоаналіз афінної біграмної підстановки

Мета роботи: Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Виконали: студент та студентка групи ФБ-42
М`якенький Антон та Кисельова Єлизавета

Є шифртекст, отриманий афінною підстановкою біграм російського тексту. Текст не має пробілів, розділових знаків і великих літер. Літера «ё» замінена на «е», а «ъ» на «ь». Алфавіт складається з m = 31 літери, які пронумеровані від 0 до 30.

Кожна біграма (x_1, x_2) кодується числом $X = x_1 \cdot 31 + x_2$, що лежить від 0 до 960. Шифрування виконується за формулою $Y = (a \cdot X + b) \bmod 961$, а розшифрування за формулою $X = a^{-1} \cdot (Y - b) \bmod 961$. Ключ складається з пари (a, b), де a взаємно просте з 961.

Завдання: знайти ключ і розшифрувати текст за допомогою частотного аналізу біграм.

Варіант: **10**. Файл variant_10.txt, довжина шифртексту 7610 літер (3805 біграм).

1. Математичні функції.

- inverse_by_modulo(a, m). Обчислює обернений елемент за модулем розширеним алгоритмом Евкліда. Якщо $\gcd(a, m) \neq 1$, повертає None.

```python
def inverse_by_modulo(a, m): #обчислює обернений елемент за розширеним алгоритмом Евкліда
    a %= m
    r, r_next = a, m
    u, u_next = 1, 0
    while r_next:
        q = r // r_next
        r, r_next = r_next, r - q * r_next
        u, u_next = u_next, u - q * u_next
    if r != 1:
        return None
    return u % m
```

- solve_linear_comparison(a, b, n). Розв'язує $a \cdot x \equiv b \pmod n$ за правилами з теоретичної частини. Якщо d = gcd(a, n) не ділить b, розв'язків немає.

Якщо ділить, то розв'язків рівно d: x_0, x_0+n_1, …, $x_0+(d-1)n_1$. Повертаються всі.

```python
def solve_linear_comparison(a, b, n): #розв'язує порівняння a·x ≡ b (mod n).
    a %= n
    b %= n
    d = gcd(a, n)
    if b % d != 0:
        return []
    a1, b1, n1 = a // d, b // d, n // d
    x0 = (b1 * inverse_by_modulo(a1, n1)) % n1 if n1 > 1 else 0
    return [(x0 + i * n1) % n for i in range(d)]
```

2. Частоти біграм

Шифртекст ділиться на біграми, що не перетинаються. Підраховуються частоти, і береться 5 найчастіших:

- read_text (читання файлу, відкидання зайвих символів)

```python
def read_text(path):
    raw = open(path, encoding='utf-8').read().lower()
    return ''.join(c for c in raw if c in ALPHABET)
```

- top_bigrams (біграми, що не перетинаються, і частоти)

```python
def top_bigrams(text, n=5): #знаходить п'ять найчастіших біграм шифртексту
    dist = {}
    for i in range(0, len(text) - 1, 2):
        bg = text[i:i + 2]
        dist[bg] = dist.get(bg, 0) + 1
    return sorted(dist, key=dist.get, reverse=True)[:n]
```

Вивід (за допомогою python lab3.py variant10.txt):

```
Файл: variant10.txt, довжина шифртексту: 7610
Найчастіші біграми мови:      ['ст', 'но', 'то', 'на', 'ен']
Найчастіші біграми шифртексту: ['сг', 'жэ', 'ям', 'нг', 'тм']
```

3. Перебір співставлень

Береться кожна впорядкована пара (X*, X**) із п'яти біграм мови (ст, но, то, на, ен) і кожна впорядкована пара (Y*, Y**) із п'яти найчастіших біграм шифртексту. Для

кожного розв'язується система (1): із рівняння (2) знаходиться a, потім b = (Y* − a·X*) mod 961.

- find_keys (система (1): розв'язує рівняння (2), знаходить b, відкидає a, не взаємно прості з 961)

```python
def find_keys(X1, X2, Y1, Y2): #розв'язує систему для кожної знайденої пари
    keys = []
    for a in solve_linear_comparison(X1 - X2, Y1 - Y2, M2):
        if gcd(a, M2) != 1:          # a має бути взаємно простим з m^2
            continue
        b = (Y1 - a * X1) % M2
        keys.append((a, b))
    return keys
```

- Цикл у attack(), а конкретніше два рядки з permutations(range(5), 2), вони перебирають усі пари.

```python
def attack(ciphertext, clear_top):
    cipher_top = top_bigrams(ciphertext, 5)
    print("Найчастіші біграми мови:     ", clear_top)
    print("Найчастіші біграми шифртексту:", cipher_top)

    clear = [bigram_to_num(b) for b in clear_top]
    cipher = [bigram_to_num(b) for b in cipher_top]

    found = []
    seen = set()
    for i, j in permutations(range(5), 2): #перебирають усі пари біграм мови і всі пари біграм шифртексту
        for k, l in permutations(range(5), 2):
            for (a, b) in find_keys(clear[i], clear[j], cipher[k], cipher[l]):
                if (a, b) in seen:
                    continue
                seen.add((a, b))
                text = decrypt(a, b, ciphertext)
                if is_meaningful(text):
                    found.append((score(text), a, b, text,
                                  (clear_top[i], clear_top[j], cipher_top[k], cipher_top[l])))
    print(f"Перевірено кандидатів на ключ: {len(seen)}")
    found.sort(reverse=True)
    return found
```

Вивід:

```
Перевірено кандидатів на ключ: 261
```

Саме стільки унікальних ключів дав перебір

4. Розшифрування і перевірка розпізнавачем

Розпізнавач приймає текст, лише якщо виконуються три умови одночасно:

1. Частота часто вживаних літер о, а, е, и, н, т більша за 0,38.
2. Частота рідкісних літер ф, щ, ь менша за 0,06.

3. Частка заборонених біграм менша за 0,01. У списку зокрема: жы, шы, щы, чы, гы, кы, хы, ыы, ьь, ьы, ьй, ээ і сполучення м'якого знака з голосними

У природній мові частоти літер стабільні й нерівномірні. Часті літери становлять майже половину тексту, а рідкісні мало. Неправильний ключ дає текст, схожий на випадкові символи: літери там розподілені майже рівномірно, часті літери трапляються рідше, рідкісні частіше, а заборонені біграми з'являються постійно. Критерій простий, швидкий і ґрунтується на властивостях, які методичка радить перевіряти.

- decrypt (формула $X = a^{-1}(Y - b) \bmod 961$)

```
def decrypt(a, b, text):  #розшифровує шифртекст за формулою X = a⁻¹(Y − b) mod m²
    a_inv = inverse_by_modulo(a, M2)
    out = []
    for i in range(0, len(text) - 1, 2):
        y = bigram_to_num(text[i:i + 2])
        out.append(num_to_bigram((a_inv * (y - b)) % M2))
    return ''.join(out)
```

- FORBIDDEN, COMMON, RARE

```
FORBIDDEN = ['аь', 'оь', 'уь', 'ыь', 'эь', 'юь', 'яь', 'еь', 'иь', 'ьь', 'ьы', 'ьй',
             'жы', 'шы', 'щы', 'чы', 'гы', 'кы', 'хы', 'йь', 'ыы', 'ээ', 'ою', 'яя',
             'щц', 'цщ', 'жщ', 'шж', 'фщ', 'щф', 'ьа', 'ьэ', 'ьо']
COMMON = 'оаеинт'
RARE = 'фщь'
```

- is_meaningful

```
def is_meaningful(text): #перевіряє частоту часто вживаних літер, частоту р
    n = len(text)
    common = sum(text.count(c) for c in COMMON) / n   # у рос. мові ~0.45
    rare = sum(text.count(c) for c in RARE) / n       # ~0.02-0.03
    forb = sum(text.count(f) for f in FORBIDDEN) / (n - 1)
    return common > 0.38 and rare < 0.06 and forb < 0.01
```

- score

```
def score(text):
    n = len(text)
    common = sum(text.count(c) for c in COMMON) / n
    forb = sum(text.count(f) for f in FORBIDDEN) / (n - 1)
    return common - 5 * forb
```

Вивід:

```
Ключ: a=300, b=400  (X*=ст, X**=но -> Y*=сг, Y**=тм), score=0.465
поздновечеромнаверандесиделколяичтотописалвтемнотебумагуитутолкомнельзябылоразглядетьвремяотвременионвосклицалаг
ерьчутьстукнулаточновсеткуотмоскитовудариласьночнаябабочкалинашепнулауфманонаселярядомснимнакачеливоднойночной
```

Контрольні питання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?

Афінна підстановка біграм зберігає частоти біграм, бо одна й та сама біграма відкритого тексту завжди шифрується в одну й ту саму біграму шифртексту. Тому найчастіші біграми мови (ст, но, то, на, ен) відповідають найчастішим біграмам шифртексту (сг, жэ, ям, нг, тм)

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?

Перша: припущення про відповідність біграм може бути хибним, тому ключ формально існує, але дає незрозумілий текст. Друга: ключ допустимий, лише якщо a взаємно просте з $961 = 31^2$. Серед розв'язків рівняння (2) були такі, де a ділиться на 31, і їх треба відкидати, бо обернений елемент не існує.

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?

Так. Із 400 співставлень 32 не мали розв'язків (b не ділилось на d), а 8 мали кілька розв'язків. Це випадки, коли $\gcd(X^*-X^{**}, 961) = 31$, і тоді рівняння має 31 розв'язок. Усі розв'язки повертаються, потім відкидаються ті, де a не взаємно просте з 961, а решта розшифровується й перевіряється розпізнавачем.

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?

Після відкидання повторів було 261 унікальних кандидатів. Тільки один із них (a = 300, b = 400) дав текст з частотами літер, типовими для російської мови, і майже без заборонених біграм. Розшифрований текст складається зі нормального тексту, і це підтверджує ключ.

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його?

Комбінацію частот часто вживаних літер, рідкісних літер і заборонених біграм. Обрано її, бо вона проста в реалізації, швидка (один прохід по тексту) і добре розділяє мову та шум. Методичка прямо радить перевіряти частоти літер.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами?

Для правильного ключа частка літер «оаеинт» становила 0,473, рідкісних літер 0,024, заборонених біграм 0,0017: всі три умови виконані. Для неправильних ключів частка часто вживаних літер була в діапазоні 0,162–0,357 (середнє 0,244), рідкісних 0,039–0,150, заборонених біграм 0,012–0,064. Жоден із 260 неправильних ключів не пройшов усі три перевірки.